LAB 7 REPORT

Fuzzing for Input Validation Bugs

Student	Chiranth J Chigateri
Course / Lab	Lab 7 – Python Fuzz Testing
Testing tools	Python, pytest, Hypothesis
Execution date	10 November 2026

1. Objective

The objective of this lab is to use fuzz testing with the Hypothesis library to identify input-validation defects in the supplied transit-processing program, analyse the failures, and improve the robustness of its functions through defensive input validation.

2. Functions Under Test

parse_transit_corridor(corridor_code) – parses a corridor code, extracts origin, destination, and zone count, then calculates and formats the fare.
lookup_concession_discount(category_name) – looks up the multiplier associated with a concession category.
compute_passenger_density(occupancy_ratio_str) – parses passengers and kilometres and calculates passenger density.

3. Fuzz Testing Approach

The test suite uses Hypothesis' **@given** decorator and strategies such as **st.text()** and **st.one_of(st.text(), st.none())** to generate a range of unexpected and malformed inputs. The three safety tests verify that invalid inputs are rejected using ValueError rather than leaking unexpected exceptions. Additional property-based tests exercise valid corridor values, supported concession categories, and numeric passenger-density inputs.

4. Defect Observed and Corrective Work

The initial fuzz run exposed a failure in concession lookup: an empty or unsupported category reached the dictionary lookup and raised **KeyError**. The application was updated to validate input types and formats before processing, reject unsupported concession categories explicitly, validate corridor structure and positive zone counts, and validate the passenger/km input before division. Invalid inputs are reported as **ValueError**.

5. Final Test Results

Test	Result
test_parse_transit_corridor_safe	PASSED
test_lookup_concession_discount_safe	PASSED

Test	Result
test_compute_passenger_density_safe	PASSED
test_valid_corridor	PASSED
test_valid_concession_categories	PASSED
test_valid_passenger_density	PASSED
Overall	**6 passed**

The final recorded pytest runs completed successfully with six tests passing (0.36 s and 0.37 s in the two supplied captures).

6. Application Verification

The fixed application was executed with the following sample inputs: corridor **DT-UP-3**, concession category **STUDENT**, and passenger/km input **50/10.0**. The recorded outputs were: route DT->UP, 3 zones, fare **$4.75**; concession multiplier **0.5**; and passenger density **5.0**.

7. Conclusion

Hypothesis-generated inputs exposed an invalid concession-category handling bug. After adding defensive validation to the three processing functions, the final test suite passed all six tests and the fixed program produced the expected results for the sample inputs. This demonstrates how property-based fuzzing can reveal edge cases that may not be covered by a small set of manually selected examples.

8. Screenshot Evidence

The following screenshots document the initial failure, successful execution of the fixed application, and the two final passing test runs.

Screenshot Evidence 1

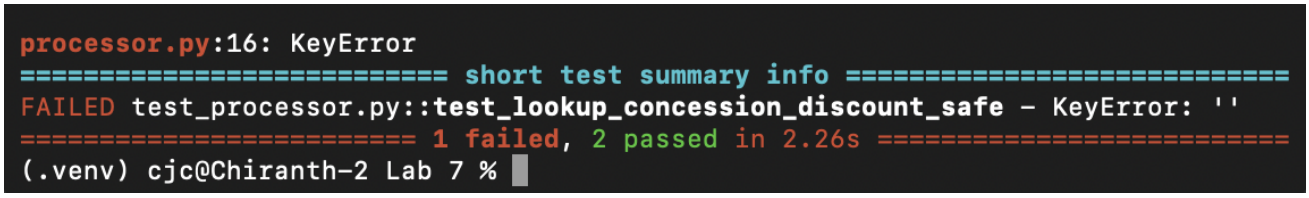


Figure 1. Initial fuzz-test failure: invalid concession input triggered KeyError.

Screenshot Evidence 2

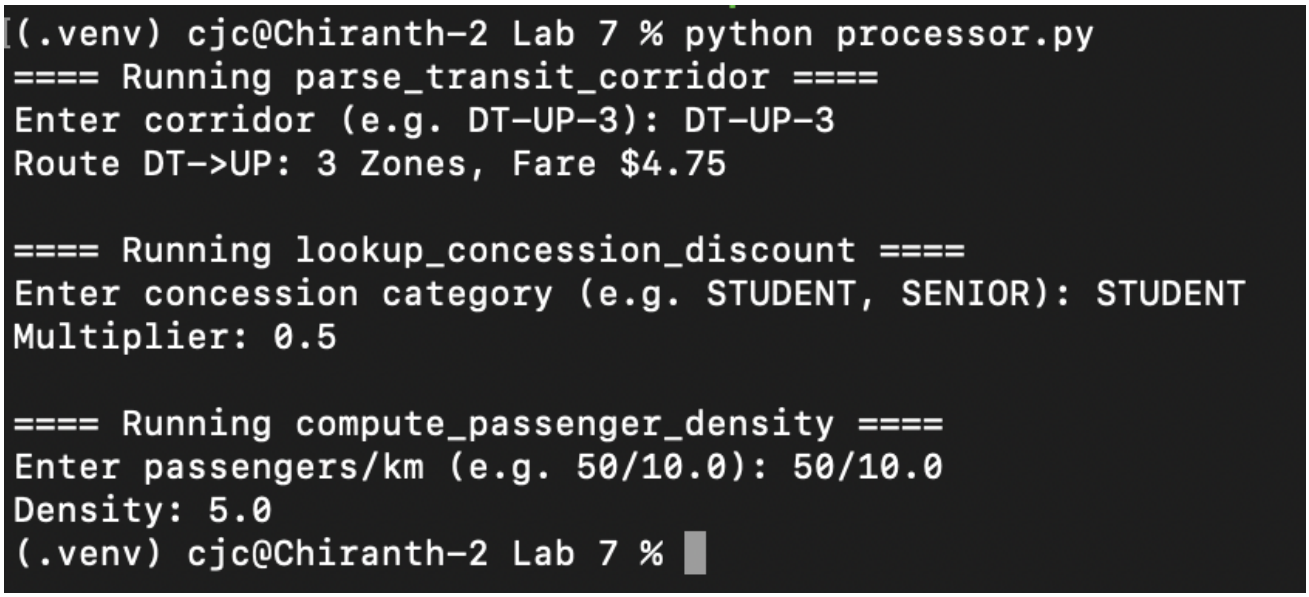


Figure 2. Fixed application execution using DT-UP-3, STUDENT, and 50/10.0.

Screenshot Evidence 3

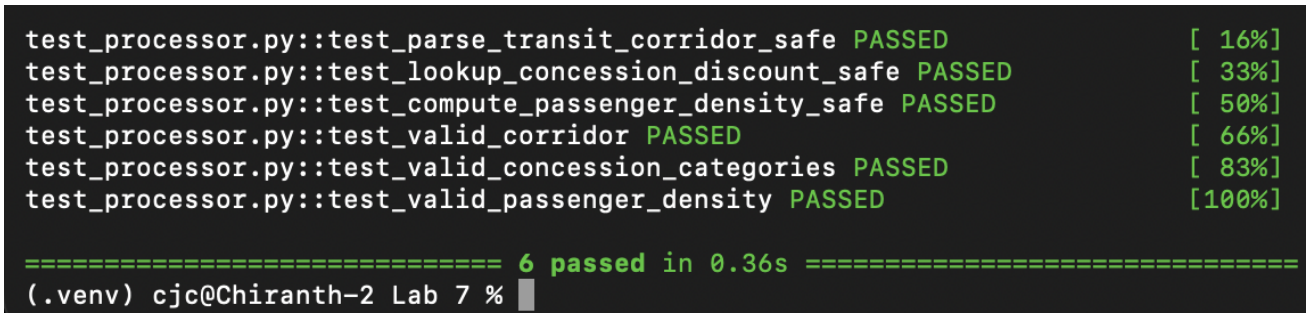


Figure 3. Hypothesis test suite: all six tests passed.

Screenshot Evidence 4

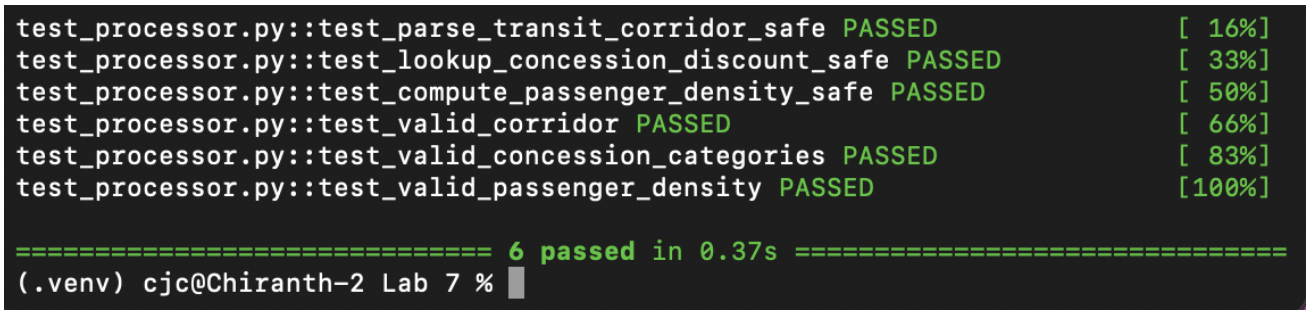


Figure 4. Final verification capture: all six tests passed again.